CHURNCE

Real-Time Customer Churn Intelligence & Lifecycle Optimization Platform

COMPLETE PROJECT DOCUMENTATION

AI-powered browser extension • Live evidence dashboard • Real-time RAG • Lifecycle intelligence • Resolution-to-reengagement

Understand the friction. Improve the experience. Bring customers back.

Project type: AI-powered browser extension and intelligence platform
AI technologies: Google Gemma and selected NVIDIA AI services
Primary interface: Browser extension plus live intelligence dashboard
Document status: Developer-ready project specification

Prepared as a technical foundation for the Churnce hackathon prototype.

Contents

- 1. Executive Summary
- 2. Problem Statement
- 3. Project Objectives
- 4. Intended Users
- 5. System Overview
- 6. Browser Extension
- 7. Event Normalization and Semantic Chunking
- 8. The Ten-Second Review Gate
- 9. Real-Time Retrieval-Augmented Generation
- 10. Google Gemma Intelligence Engine
- 11. Customer Lifecycle Intelligence
- 12. Root-Cause Intelligence
- 13. Next-Best-Action Engine
- 14. Resolution-to-Reengagement Engine
- 15. Feedback and Learning System
- 16. Live Dashboard Specification
- 17. Technology Stack
- 18. Proposed Repository Structure
- 19. Data Model
- 20. API Design
- 21. Security, Privacy and Governance
- 22. Evaluation Strategy
- 23. Hackathon Implementation Roadmap
- 24. Demonstration Scenario
- 25. Known Limitations
- 26. Future Development
- 27. Expected Outcomes
- 28. Conclusion
- 29. Reference Documentation

1. Executive Summary

Churnce is an AI-powered customer intelligence platform designed to help businesses understand why customers disengage, identify obstacles in their digital experiences, recommend actionable improvements, and encourage customers to return after those improvements have been verified.

The platform operates through a browser extension installed on an authorized business website. The extension captures meaningful customer interaction events, such as navigation patterns, repeated failed actions, response delays, abandoned workflows and unsuccessful searches.

These events are normalized, filtered for sensitive information and organized into meaningful evidence chunks. Each chunk is presented in a live dashboard with a ten-second editing window before it is ingested into the retrieval-augmented generation (RAG) system.

The RAG pipeline retrieves relevant customer activity, product documentation, support records, application logs, feature information and historical intervention outcomes. Google Gemma interprets the retrieved evidence and generates structured insights, while supported NVIDIA AI services can provide embeddings and reranking.

Churnce then identifies potential causes of customer friction and recommends appropriate interventions. These may include engineering tickets, user-interface improvements, onboarding changes, support escalations and lifecycle marketing campaigns.

When a business resolves an issue, Churnce verifies the improvement using available evidence and prepares customer-facing communications that emphasize the improved experience rather than unnecessarily exposing technical problems.

All stages are observable through a live dashboard, from event collection and chunk review to retrieval, diagnosis, recommendation, approval, execution and feedback.

1.1 Core value proposition

Churnce connects customer behavior, product quality and lifecycle marketing in a single intelligence workflow. Instead of treating customer churn exclusively as a marketing problem, it investigates whether customers are leaving because of technical failures, poor usability, missing functionality, slow responses, confusing onboarding or unresolved support issues.

It helps businesses move from reactive customer retention to evidence-driven customer experience improvement.

2. Problem Statement

Businesses frequently collect customer interaction data without connecting it to the underlying reasons for customer disengagement.

Marketing teams may respond to declining engagement with more emails, discounts or promotional offers. However, these interventions can fail when the real problem is a broken checkout button, a slow dashboard, an incomplete onboarding flow or a missing product feature.

Several challenges contribute to this problem:

- Customer activity is distributed across analytics tools, application logs, support systems and marketing platforms.
- Traditional analytics often report what happened without explaining why it happened.
- Product and engineering teams may not recognize the customer-retention impact of usability or reliability issues.
- Generic marketing campaigns may ignore the specific obstacle experienced by each customer segment.
- Businesses may continue promoting a product even when an unresolved issue makes the experience frustrating.
- Product improvements are not always connected to subsequent customer re-engagement and retention outcomes.

Churnce addresses these challenges by creating a continuous evidence-driven feedback loop between customer behavior, product improvement and lifecycle marketing.

3. Project Objectives

The primary objectives of Churnce are to:

- Monitor meaningful customer interaction events on authorized websites.
- Convert raw events into structured, understandable evidence chunks.
- Provide a configurable ten-second review window before RAG ingestion.
- Analyze activity in near real time using retrieval-augmented generation.
- Connect observed behavior with relevant product knowledge and historical evidence.
- Identify potential UX, responsiveness, feature-gap, onboarding and support issues.
- Generate evidence-backed recommendations with confidence and uncertainty indicators.
- Suggest the most appropriate next action for each identified problem.
- Generate context-aware lifecycle marketing emails.
- Verify product improvements before using them in customer-facing claims.
- Measure whether recommended interventions improve customer outcomes.
- Provide transparent, live visualization of the entire processing pipeline.

3.1 Success criteria

The initial implementation should demonstrate that:

- Events appear in the dashboard after collection.
- Pending chunks can be edited, redacted, approved or discarded.
- Only validated and approved chunks enter the searchable knowledge base.
- Relevant evidence can be retrieved and inspected.
- The AI produces recommendations linked to supporting evidence.
- Email drafts are based on verified facts and appropriate customer context.
- Actions require the appropriate approval before execution.
- Outcomes can be measured against a baseline or comparison group.

The hackathon version will demonstrate these capabilities using a controlled website and synthetic customer journeys before attempting deployment on real business websites.

4. Intended Users

4.1 Business owners and growth teams

Use Churnce to understand customer friction, prioritize improvements and develop more relevant retention strategies.

4.2 Product managers and UX designers

Use evidence-backed findings to identify confusing workflows, missing features and usability problems.

4.3 Software developers and engineering teams

Use detected failures, application logs and correlated customer activity to investigate technical issues and prioritize fixes.

4.4 Customer support teams

Use reconstructed customer journeys to understand recurring complaints and prepare appropriate follow-ups.

4.5 Marketing teams

Use verified product improvements and customer lifecycle context to prepare relevant re-engagement campaigns.

5. System Overview

Churnce consists of seven major subsystems.

- Browser extension and event collector.
- Event normalization and privacy filtering.
- Chunking and ten-second review gate.
- Real-time RAG and knowledge retrieval.
- AI intelligence and lifecycle analysis.
- Next-best-action and execution engine.
- Live dashboard and outcome feedback system.

5.1 High-level architecture

6. Browser Extension

6.1 Purpose

The browser extension is the primary observation component. It is installed in a browser and configured for a website that the business owns or is authorized to monitor. Its responsibility is to collect meaningful interaction events and send normalized data to the Churnce backend.

The extension should not attempt to capture every action indiscriminately. It should collect only events needed to understand the customer journey.

6.2 Supported events

- Page navigation and route changes.
- Page load and response timing.
- Clicks on designated interactive elements.
- Repeated clicks or failed interactions.
- Form validation failures without recording sensitive field values.
- Search queries after privacy filtering.
- Search results with no useful matches.
- Abandoned checkout or onboarding steps.
- Repeated attempts to complete the same task.
- Visible application error messages.
- Completion of designated business workflows.
- Session duration and inactivity intervals.

Application-specific instrumentation may provide more accurate events than generic browser observation. Where available, explicit application events should be preferred.

6.3 Example event

```
{
  "event_id": "evt_000184",
  "session_id": "session_042",
  "event_type": "interaction_failure",
  "page_route": "/checkout",
  "element_category": "primary_action",
  "event_timestamp": "2026-10-27T10:15:30Z",
  "response_time_ms": 4800,
  "outcome": "timeout",
  "consent_state": "authorized"
}
```

This event describes a failed interaction without storing a password, payment credential or complete form submission.

6.4 Extension architecture

The extension should use Chromium Manifest V3 with content scripts for authorized page observation, a background service worker for messaging and event delivery, a settings interface for monitoring configuration, a consent and monitoring-status indicator, and a secure communication channel with the backend.

The extension should not contain Gemma or NVIDIA API keys. AI inference and credential handling belong on the backend.

6.5 Privacy controls

- Require explicit authorization for monitored websites and relevant data collection.
- Avoid recording passwords, payment information and sensitive form values.
- Avoid unrestricted keystroke logging, hidden surveillance and unnecessary browsing history collection.
- Filter or redact personally identifiable information before transmission.
- Support monitoring disablement and data retention controls.
- Transmit events only to the configured Churnce backend.
- Enforce access controls for customer-level information.

Where consent or applicable law requires additional authorization, the system must obtain it before collection.

7. Event Normalization and Semantic Chunking

7.1 Purpose

Raw events are often too granular to provide useful context to a language model. For example, three button clicks, two error messages and one navigation event are more meaningful when combined into a single failed-checkout episode.

Churnce therefore transforms raw events into semantic chunks representing meaningful customer activities.

7.2 Chunking strategies

- Time-based chunking: group related events within a configurable interval.
- Journey-based chunking: group events belonging to a workflow, such as registration, checkout or onboarding.
- Error-based chunking: group events around a repeated failure or error condition.
- Semantic chunking: group related events according to their meaning and relationship.

A hybrid strategy is recommended. Journey boundaries define the main context, while time and event-count thresholds control chunk size.

7.3 Example chunk

```
{
  "chunk_id": "chk_000087",
  "session_id": "session_042",
  "journey_stage": "checkout",
  "summary": "Repeated checkout interaction failures",
  "events": [
    "Primary checkout action attempted",
    "Request timed out",
    "Checkout action attempted again",
    "Second request returned an error"
  ],
  "observations": {
    "attempt_count": 3,
    "observed_response_time_ms": 4800,
    "workflow_completed": false
  },
  "sensitivity_status": "filtered",
  "review_status": "pending",
  "version": 1
}
```

7.4 Chunk metadata

- A unique identifier.
- Session or pseudonymous journey identifier.
- Source event identifiers.
- Event timestamps.
- Journey stage.
- Chunk creation timestamp.
- Redaction and validation status.
- Review status.
- Version number.
- Ingestion status.
- Links to supporting evidence, where permitted.

7.5 Chunk lifecycle

The system should use versioned updates and idempotent ingestion to prevent duplicate chunks and stale edits.

8. The Ten-Second Review Gate

8.1 Purpose

The review gate gives the operator a short opportunity to inspect and modify the evidence before it becomes part of the searchable RAG knowledge base. It is a defining feature of Churnce because it makes the ingestion process visible and controllable.

8.2 Workflow

At T+0 seconds

A new chunk appears in the dashboard with its summary, event sequence, source references and privacy-filtering status.

From T+0 to T+10 seconds

The operator can edit the chunk summary, correct misleading descriptions, redact unnecessary information, approve the chunk immediately, discard the chunk or inspect the source events.

At T+10 seconds

If no editing operation is active, the system proceeds to validation and ingestion according to the configured review policy.

After validation

Create embeddings, index the chunk and associate it with its session, timestamp and evidence sources.

After retrieval

Run the intelligence analysis, display the retrieved evidence and recommendations, and allow the operator to approve the next action.

8.3 Important implementation distinction

The ten-second interval is a review buffer, not a guarantee that an operator will always be available. For a controlled hackathon demonstration, an inactivity timeout may automatically approve a privacy-filtered chunk for further validation. For real customer data, configurable manual-approval or restricted-ingestion policies should be available.

Chunks containing sensitive data or failed validation checks must never enter the vector index merely because the timer expires.

8.4 Dashboard controls

- Countdown timer.
- Editable summary.
- Event timeline.
- Redaction indicator.
- Approve, save, discard and inspect actions.
- Current processing state.
- Ingestion result.

The timer must be implemented by the backend or a reliable job scheduler, rather than depending exclusively on the browser dashboard remaining open.

9. Real-Time Retrieval-Augmented Generation

9.1 Purpose

The RAG system provides the evidence used by the intelligence engine. It prevents the model from relying solely on a raw stream of events or inventing explanations without consulting relevant business information.

Churnce uses RAG to retrieve customer journey evidence, product documentation, application logs, support tickets, feature specifications, policies and historical intervention outcomes.

9.2 Knowledge collections

A. Customer journey evidence

Approved, privacy-filtered chunks representing customer behavior.

B. Product knowledge

Product documentation, feature specifications, onboarding instructions, release notes and known limitations.

C. Engineering evidence

Authorized application logs, error reports, issue descriptions and release information.

D. Support evidence

Appropriately redacted support tickets, recurring complaints and approved resolution records.

E. Intervention memory

Previous recommendations, approved actions, execution records and observed outcomes.

F. Business policies

Communication rules, consent requirements, escalation procedures, retention policies and approved claims.

Access controls must be applied before retrieval. A model must not gain access to another customer's private information simply because the information is semantically relevant.

9.3 RAG ingestion pipeline

The system should retain source identifiers and timestamps so that retrieved evidence can be traced back to its origin.

9.4 Retrieval pipeline

When an issue is detected, Churnce constructs a retrieval query from the current evidence and the relevant business question. For example: "Find previous checkout failures, related error logs, known payment issues, relevant product documentation and verified interventions associated with incomplete checkout journeys."

- Search the relevant knowledge collections.
- Apply tenant, session and permission filters.
- Retrieve semantically similar passages.
- Use reranking to prioritize the most relevant evidence.
- Remove duplicates and outdated or conflicting records where appropriate.
- Assemble a bounded evidence package for Gemma.
- Preserve source references for inspection.

9.5 Suggested NVIDIA integration

Use NVIDIA embedding services to convert text into vectors and NVIDIA reranking services to refine candidate evidence, subject to the selected model's availability and API compatibility. PostgreSQL with pgvector can store embeddings, metadata and source references.

NVIDIA services should be selected according to the actual operations they support. An embedding endpoint is not a substitute for a generative reasoning model, and a reranking endpoint does not independently diagnose a customer problem.

9.6 Why this is more than a chatbot

The RAG system is continuously updated with approved evidence. It is used to investigate structured events and drive workflows rather than merely answer questions. The intelligence engine must cite retrieved evidence, distinguish observed facts from hypotheses, and identify when there is insufficient evidence for a confident diagnosis.

10. Google Gemma Intelligence Engine

10.1 Responsibilities

- Classifying the customer lifecycle stage.
- Summarizing retrieved evidence.
- Identifying recurring patterns.

- Proposing possible causes of customer friction.
- Distinguishing symptoms from likely root causes.
- Identifying feature requests and UX problems.
- Generating structured recommendations.
- Preparing lifecycle marketing drafts.
- Explaining the rationale behind each proposed action.

10.2 Input structure

The model should receive a bounded, structured context containing the current incident or customer journey, relevant retrieved evidence, source references, known product behavior, verified engineering information, applicable business policies, historical interventions and outcomes, and the required output schema.

The model must treat retrieved documents and customer-provided text as untrusted evidence, not as instructions that override system policy.

10.3 Output structure

```
{
  "incident_type": "checkout_friction",
  "lifecycle_stage": "conversion",
  "observed_facts": [
    "Three checkout attempts were recorded",
    "Two requests returned errors"
  ],
  "hypotheses": [
    {
      "description": "A checkout reliability issue may be preventing completion",
      "confidence": 0.84
    }
  ],
  "evidence_ids": [
    "chk_000087",
    "log_000219"
  ],
  "recommended_action": {
    "type": "engineering_investigation",
    "priority": "high",
    "reason": "Repeated failures affect an important conversion workflow"
  },
  "customer_message_eligible": false
}
```

The confidence value is a model-produced estimate, not a calibrated probability. Churnce should evaluate confidence against historical labeled incidents before relying on it for automated decisions.

10.4 Model output validation

All model output must pass schema validation. The system should reject malformed recommendations, unknown action types, unsupported claims and actions that violate policy. Gemma proposes actions. The deterministic action engine decides whether those actions are permitted.

11. Customer Lifecycle Intelligence

Churnce uses customer lifecycle stages to interpret behavior and select relevant interventions.

11.1 Awareness

The customer discovers the business or product. Potential issues include unclear value propositions, poor landing-page performance and difficulty finding relevant information.

11.2 Acquisition

The customer registers, subscribes or begins a purchase. Potential issues include registration failures, payment friction and confusing account creation.

11.3 Activation

The customer attempts to experience the product's core value. Potential issues include incomplete onboarding, missing guidance and inaccessible features.

11.4 Engagement

The customer uses the product repeatedly. Potential issues include declining usage, poor search results, slow workflows and missing functionality.

11.5 Retention

The customer continues using or purchasing the product. Potential issues include unresolved complaints, reliability problems and declining engagement.

11.6 Reactivation

A previously active customer returns after a period of inactivity. Potential interventions include relevant feature announcements, assistance with an interrupted workflow and benefit-focused reminders.

11.7 Advocacy

A satisfied customer may recommend the product or share positive feedback. Advocacy requests should be based on appropriate eligibility and consent rather than indiscriminate solicitation.

11.8 Lifecycle stage assignment

Churnce should not infer a customer's complete lifecycle stage from a single click. It should combine previous activity, journey completion, frequency of use, time since last meaningful interaction, product adoption, support history and relevant purchase or subscription events, where authorized.

The lifecycle classifier should support an unknown or uncertain state when evidence is insufficient.

12. Root-Cause Intelligence

12.1 Objective

Churnce should identify the most plausible explanations for customer friction, rather than automatically assuming that inactivity means a customer is about to leave.

12.2 Issue categories

- UI and usability friction.
- Slow response or poor performance.
- Application errors.
- Missing features.
- Search and discovery problems.
- Onboarding difficulties.
- Payment or checkout failures.
- Documentation gaps.
- Repeated unresolved support issues.

- Declining product engagement.
- Communication fatigue.
- Unknown or insufficiently supported causes.

12.3 Cross-source correlation

Suppose multiple customers abandon onboarding at the same step. Churnce can retrieve their approved interaction chunks, related application error logs, onboarding documentation, existing engineering tickets and previous interventions addressing similar problems.

If these sources consistently indicate a failing API request, the system can recommend investigating the endpoint rather than sending additional promotional emails.

The recommendation should distinguish correlation from causation. Confirming that the issue caused abandonment may require application tests, engineering verification or controlled experiments.

13. Next-Best-Action Engine

13.1 Purpose

The next-best-action engine converts AI recommendations into controlled business workflows. It decides which actions are eligible, which action should be prioritized, and whether human approval is required.

13.2 Supported action categories

Engineering actions

- Create an issue report.
- Prepare a bug reproduction summary.
- Recommend performance investigation.
- Propose a UX improvement.
- Suggest a missing feature for product review.

Support actions

- Prepare a support escalation.
- Summarize a customer journey.
- Recommend a knowledge-base improvement.

Lifecycle marketing actions

- Prepare a re-engagement email.
- Draft an onboarding reminder.
- Prepare a feature announcement.
- Suggest an appropriate recovery communication.
- Recommend suppressing a campaign until an issue is resolved.

Analytical actions

- Track a metric before and after an intervention.
- Identify a segment for further investigation.
- Recommend an experiment.
- Compare historical interventions.

13.3 Action ranking

A proposed action can be ranked using severity of the observed problem, number of affected journeys, strength and relevance of supporting evidence, business impact, estimated implementation effort, customer harm if the problem remains unresolved, confidence in the diagnosis, and compliance and communication eligibility.

A transparent scoring model is preferable to allowing the language model to invent its own priorities without constraints.

13.4 Action lifecycle

Actions must be idempotent where possible, and retries must not create duplicate emails or engineering tickets.

14. Resolution-to-Reengagement Engine

14.1 Purpose

The Resolution-to-Reengagement Engine connects verified product improvements with relevant customer lifecycle communications. Its purpose is to encourage customers to return by highlighting the value of the improved experience, rather than unnecessarily exposing internal technical incidents.

For example, if a slow checkout interaction has been fixed, Churnce may prepare an email inviting an eligible customer to continue shopping. The message emphasizes convenience and completing the customer's intended task, not the engineering defect itself.

14.2 Resolution verification

Before a resolved issue can trigger a customer-facing claim, Churnce must establish that the fix is supported by evidence.

- An approved engineering resolution.
- A deployed release associated with the issue.
- Successful automated regression tests.
- Improved application telemetry.
- A successful test of the affected workflow.
- Confirmation that the improvement is available to the intended customer segment.

The system should distinguish a fix being deployed from the customer-facing experience actually improving.

14.3 Resolution record

```
{
  "resolution_id": "res_000031",
  "incident_id": "inc_000012",
  "issue_category": "checkout_responsiveness",
  "resolution_status": "verified",
  "verification_sources": [
    "release_000014",
    "test_run_000029"
  ],
  "verified_at": "2026-10-27T12:00:00Z",
  "approved_claims": [
    "The checkout flow has been improved"
  ],
  "email_eligible": true
}
```

The example represents a hypothetical record. A production implementation must verify each source and claim before using it.

14.4 Benefit-focused email generation

The email generator receives the customer's eligible lifecycle context, the task the customer attempted, the verified improvement, approved customer-facing claims, brand tone and messaging rules, communication consent and suppression status, and the desired next action.

Gemma prepares a natural message based on those inputs.

Example: checkout improvement

Subject: Your next checkout could be easier

Hi there,

We're always working to make your shopping experience smoother. When you're ready, come back and pick up where you left off.

Your next purchase is waiting, and we'd love to make getting there feel effortless.

Continue shopping.

Thanks for being with us.

This message does not directly describe the technical defect. It communicates a customer benefit and provides a relevant path back to the product.

14.5 Other examples

Verified improvement	Potential message angle
Faster dashboard	Get back to your work with a smoother experience
Clearer onboarding	Pick up where you left off and continue getting started
Improved product search	Discover products more easily
New export feature	Explore a more convenient reporting workflow
Resolved payment retry problem	Return to complete your intended purchase

The wording must remain truthful. Churnce must not claim a feature is available to a customer until its availability has been verified.

14.6 Communication controls

- Valid communication consent and legal basis.

- Customer eligibility.
- Unsubscribe and suppression status.
- Contact-frequency limits.
- Duplicate campaign prevention.
- Accuracy of the proposed claims.
- Whether the underlying issue remains unresolved.
- Whether the email is promotional or transactional.
- Applicable requirements for that message category.

A recovery email must not be used to disguise an unrelated promotional campaign or bypass marketing-consent requirements.

14.7 Email execution

The first version should generate drafts and require human approval. A later version can integrate an authorized email provider to send approved messages, record delivery status and collect available outcome events.

Sending must be controlled by the backend. The AI model must never directly send arbitrary messages without passing the action and policy checks.

15. Feedback and Learning System

15.1 Purpose

Churnce records what happens after a recommendation or intervention. The system should use these records to improve future evidence retrieval, evaluation and decision-making.

15.2 Recorded outcomes

- The original incident.
- Supporting evidence.
- The recommended action.
- The action selected by the operator.
- Approval or rejection.
- Execution status.
- Time to resolution.
- Relevant before-and-after metrics.
- Email delivery, when applicable.
- Return visits and completed tasks, where measurable.
- Unsubscribes or complaints, where available.
- Evaluation results and limitations.

15.3 Intervention memory

Past interventions become searchable evidence. For example, if a particular onboarding problem has repeatedly been resolved by simplifying the instructions, Churnce can retrieve those outcomes when investigating similar cases.

However, a successful outcome in one situation does not prove that the same intervention will work in every situation. The system should preserve the customer segment, context, action and measurement method alongside the result.

15.4 Measuring causal impact

Where practical, businesses should use randomized holdout groups or controlled experiments to determine whether a recommendation actually improves retention. Before-and-after comparisons are useful for monitoring but can be affected by seasonality, traffic changes, product releases and other factors.

Churnce should report the strength of the evidence rather than presenting every positive change as proof of causation.

16. Live Dashboard Specification

16.1 Dashboard purpose

The dashboard is the operational control room for Churnce. It demonstrates the complete system in real time and provides the interface through which operators inspect evidence, edit chunks, review recommendations and approve actions. It is not merely a collection of static analytics charts.

16.2 Dashboard modules

A. System overview

Displays events captured, chunks pending review, chunks indexed, active incidents, recommendations generated, actions awaiting approval, actions completed, processing latency and system health. Metrics must come from actual backend state rather than hardcoded demonstration values.

B. Live event stream

Displays incoming normalized events with timestamps, event categories, journey identifiers and processing status. Sensitive event contents must not be exposed unnecessarily.

C. Chunk laboratory

Displays pending chunks, their event sequences, editable summaries, countdown timers, validation status and ingestion results.

D. RAG inspector

Displays the current retrieval query, retrieved evidence passages, similarity and reranking scores, source identifiers, collection names, evidence freshness, access-control decisions and the context provided to Gemma.

E. Intelligence feed

Displays detected issues, supporting evidence, hypotheses, confidence indicators, recommended improvements and unresolved questions.

F. Action center

Displays proposed emails, engineering tickets, support tasks, approval states, execution results and action history.

G. Feedback and evaluation

Displays intervention outcomes, resolution metrics, re-engagement results and comparisons against baselines.

H. Configuration and governance

Provides monitoring configuration, data retention controls, user roles, review policies, communication rules and integration settings.

16.3 Real-time updates

The backend should publish pipeline state changes to the dashboard through WebSockets or a comparable real-time transport.

```
{
  "type": "pipeline_stage_changed",
  "chunk_id": "chk_000087",
  "previous_stage": "review",
```

```
  "current_stage": "embedding",
  "timestamp": "2026-10-27T10:15:40Z"
}
```

The dashboard should update from these backend events rather than independently guessing processing progress.

16.4 Visual design

The recommended interface uses a dark observability aesthetic with restrained teal, amber and blue accents. The design should emphasize dense but readable event information, clear stage indicators, visible review timers, evidence-linked recommendations, explicit distinction between pending, approved, failed and completed actions, minimal unnecessary navigation, accessible contrast and keyboard navigation.

17. Technology Stack

Layer	Technology	Responsibility
Browser extension	TypeScript, Chromium Manifest V3	Authorized event observation
Frontend	React, TypeScript	Live dashboard
Styling	Tailwind CSS or equivalent	Dashboard interface
Backend	Python, FastAPI	APIs and orchestration
Real-time communication	WebSockets	Live pipeline updates
Background processing	Redis-backed task queue or equivalent	Asynchronous ingestion and model calls
Database	PostgreSQL	Structured records and metadata
Vector search	pgvector	Semantic retrieval
Embeddings	NVIDIA embedding service	Vector generation
Reranking	NVIDIA reranking service	Evidence relevance refinement
Reasoning	Google Gemma through a supported API	Analysis and structured recommendations
Email execution	Authorized email service provider	Approved message delivery
Testing	Pytest and frontend testing tools	Functional and regression testing
Deployment	Docker Compose for local development	Reproducible services

The exact Gemma hosting route, NVIDIA models, API endpoints, rate limits and pricing must be verified when selecting the implementation services.

Not every component requires a separate AI model. Deterministic event processing, validation and policy checks should use conventional application code.

18. Proposed Repository Structure

```
churnce/
├── README.md
├── LICENSE
├── .env.example
├── docker-compose.yml
├── extension/
│   ├── manifest.json
│   ├── src/
│   │   ├── content/
│   │   ├── background/
│   │   ├── popup/
│   │   ├── settings/
│   │   └── events/
│   └── tests/
├── dashboard/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── services/
│   │   └── types/
│   └── tests/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   ├── collector/
│   │   ├── normalizer/
│   │   ├── chunking/
│   │   ├── review_gate/
│   │   ├── rag/
│   │   ├── llm/
│   │   ├── lifecycle/
│   │   ├── intelligence/
│   │   ├── action_engine/
│   │   ├── email/
│   │   ├── feedback/
│   │   ├── security/
│   │   └── database/
│   └── tests/
├── database/
│   ├── migrations/
│   └── schemas/
├── data/
│   ├── synthetic_events/
│   ├── product_docs/
│   └── demo_incidents/
├── docs/
│   ├── architecture.md
│   ├── api.md
│   ├── rag_pipeline.md
│   ├── privacy_and_security.md
│   ├── evaluation.md
│   └── deployment.md
└── scripts/
    ├── seed_demo.py
    └── simulate_journey.py
```

This is a proposed structure. Directories should be created as their respective modules are implemented rather than generating empty files for every planned component.

19. Data Model

19.1 Organizations

Stores tenant information, configuration and data isolation settings.

19.2 Users and roles

Stores dashboard users, roles and authorization policies.

19.3 Monitoring configurations

Stores authorized domains, event types, collection rules and retention policies.

19.4 Sessions

Stores pseudonymous journey identifiers, timestamps and lifecycle context.

19.5 Events

Stores normalized interaction events and their source metadata.

19.6 Chunks

Stores chunk content, versions, review states, validation results and ingestion status.

19.7 Evidence records

Stores indexed passages, embeddings, source references, timestamps and collection metadata.

19.8 Incidents

Stores detected problems, hypotheses, severity, supporting evidence and verification status.

19.9 Recommendations

Stores proposed actions, rationale, evidence references and approval state.

19.10 Interventions

Stores approved actions, execution results and associated measurements.

19.11 Resolutions

Stores verified fixes, supporting tests, approved customer-facing claims and resolution timestamps.

19.12 Communications

Stores email drafts, approvals, delivery status and applicable consent checks.

19.13 Outcome records

Stores measurements used to evaluate whether an intervention improved customer outcomes.

Records should be associated with tenant and access-control metadata wherever applicable. Data retention and deletion policies must also cover derived chunks and embeddings.

20. API Design

The following routes are proposed application endpoints, not existing external provider endpoints.

Method	Endpoint	Purpose
POST	/api/v1/events	Submit normalized events
GET	/api/v1/events	Retrieve authorized event records
POST	/api/v1/chunks	Create or submit a chunk
GET	/api/v1/chunks/pending	Retrieve pending review chunks
PATCH	/api/v1/chunks/{id}	Edit a pending chunk using version checks

Method	Endpoint	Purpose
POST	/api/v1/chunks/{id}/approve	Approve a chunk for validation
POST	/api/v1/chunks/{id}/discard	Discard a pending chunk
GET	/api/v1/rag/retrievals/{id}	Inspect a retrieval operation
GET	/api/v1/incidents	List authorized incidents
GET	/api/v1/incidents/{id}	Retrieve incident evidence and status
POST	/api/v1/incidents/{id}/analyze	Request an analysis
GET	/api/v1/recommendations	Retrieve recommendations
POST	/api/v1/actions/{id}/approve	Approve a proposed action
POST	/api/v1/actions/{id}/reject	Reject a proposed action
POST	/api/v1/resolutions	Record a resolution
POST	/api/v1/emails/{id}/approve	Approve an email draft
GET	/api/v1/metrics	Retrieve dashboard metrics
GET	/api/v1/stream	Establish a WebSocket connection

The backend must authenticate callers, validate authorization, enforce request limits and reject unauthorized cross-tenant access.

External Gemma and NVIDIA calls should be encapsulated in service adapters. This makes it possible to change providers without rewriting the entire application.

21. Security, Privacy and Governance

Churnce processes potentially sensitive customer behavior. Security and governance are fundamental system requirements.

21.1 Authentication and authorization

Use authenticated dashboard access, role-based permissions and tenant isolation. Possible roles include Administrator, Product analyst, Engineer, Marketing operator and Read-only reviewer. Each role should receive only the permissions required for its tasks.

21.2 Secret management

Gemma and NVIDIA API keys must be stored in backend environment variables or an appropriate secret-management service. Secrets must never be committed to source control, included in frontend bundles or embedded in the browser extension.

21.3 Data minimization

Collect only the events needed to support the declared business purpose. Avoid raw keystroke logging, passwords, payment data and unnecessary personal identifiers. Use pseudonymous identifiers and redact sensitive information before indexing.

21.4 Encryption

Use HTTPS for network communications and appropriate encryption for stored sensitive information. Restrict database access and rotate credentials according to operational policy.

21.5 Auditability

Record who approved, rejected, edited or executed an action, together with timestamps and relevant state transitions. Audit records must avoid unnecessarily duplicating sensitive customer data.

21.6 Prompt injection and untrusted evidence

Retrieved documents, application logs and customer text may contain instructions intended to manipulate the AI. Gemma must treat retrieved content as data, not as authority to override system policies. The action engine must independently validate all proposed operations.

21.7 Retention and deletion

Define retention periods for raw events, chunks, embeddings, email records and audit logs. Deleting source data must trigger appropriate cleanup of derived records and indexes where required.

21.8 Responsible automated actions

The initial implementation should require approval for external actions, including sending emails, modifying production systems and creating externally visible communications. Later automation should be introduced only for explicitly authorized, low-risk operations with appropriate monitoring and rollback controls.

22. Evaluation Strategy

Churnce must be evaluated on both technical accuracy and business usefulness.

22.1 Event collection accuracy

Measure whether expected events are captured, duplicated or missed.

22.2 Chunk quality

Evaluate whether events are grouped into coherent customer journey episodes.

22.3 Retrieval quality

Measure whether the most relevant evidence appears near the top of retrieved results. Possible metrics include Recall@K, Precision@K and ranking metrics such as NDCG.

22.4 Root-cause identification

Use synthetic incidents with known causes to measure classification accuracy, false positives and the quality of evidence supporting each diagnosis.

22.5 Recommendation quality

Have reviewers evaluate relevance, feasibility, evidence support, policy compliance and clarity.

22.6 Operational performance

Measure event-to-dashboard latency, chunk review and ingestion latency, retrieval latency, model response latency, end-to-end recommendation latency, API usage and cost, and processing failures and retry frequency.

22.7 Email effectiveness

Measure eligible delivery, bounce rates, unsubscribes, complaints, return visits and completion of the intended customer task. Email opens should not be the sole success measure because privacy features and automated clients can make them unreliable.

22.8 Business impact

Where sufficient data is available, evaluate conversion rate, onboarding completion, successful workflow completion, repeat usage, retention, reactivation and time to resolve recurring issues.

The project should not claim a reduction in customer churn until that reduction has been demonstrated with appropriate evidence.

23. Hackathon Implementation Roadmap

Phase 1: Foundation

- Create the repository.
- Configure the backend and dashboard.
- Set up PostgreSQL and pgvector.
- Define event and chunk schemas.
- Create a synthetic demonstration website.

Deliverable: A running local environment with a seeded dataset.

Phase 2: Browser extension

- Implement authorized event collection.
- Normalize and filter events.
- Deliver events to the backend.
- Display incoming events in the dashboard.

Deliverable: A live extension-to-dashboard event stream.

Phase 3: Chunking and review gate

- Implement semantic or journey-based chunking.
- Add the ten-second countdown.
- Add editing, approval and discard controls.
- Validate chunk versions and ingestion states.

Deliverable: A working, editable chunk review workflow.

Phase 4: RAG integration

- Add document ingestion.
- Integrate the selected NVIDIA embedding service.
- Implement vector search.
- Add NVIDIA reranking where available.
- Display retrieved evidence and source references.

Deliverable: An inspectable RAG pipeline.

Phase 5: Gemma intelligence

- Connect the selected Gemma API.
- Implement structured prompts and output validation.
- Add lifecycle classification.
- Generate evidence-backed incident diagnoses and recommendations.

Deliverable: A functioning AI analysis workflow.

Phase 6: Next-best-action engine

- Implement policy checks.
- Add approval workflows.
- Generate engineering tickets and support tasks.
- Prepare benefit-focused email drafts.

Deliverable: A controlled recommendation and action system.

Phase 7: Resolution and feedback

- Add verified resolution records.
- Connect improvements to eligible customer journeys.
- Generate re-engagement drafts.
- Record execution and outcome metrics.

Deliverable: A demonstrable resolution-to-reengagement loop.

Phase 8: Evaluation and presentation

- Seed reproducible incidents.
- Compare AI-assisted diagnosis with a rules-only baseline.
- Measure retrieval relevance and diagnosis accuracy.
- Test privacy and authorization controls.
- Prepare the live demonstration.

Deliverable: A measurable, end-to-end hackathon prototype.

For a short hackathon, the first six phases should take priority. Email execution and advanced learning can initially remain simulated or draft-only.

24. Demonstration Scenario

Step 1: Simulate a customer journey

A test customer visits the demonstration website, attempts checkout multiple times and encounters a slow or failing primary action.

Step 2: Observe the activity

The extension captures the permitted events and sends them to the backend.

Step 3: Create a chunk

The event processor identifies a failed-checkout episode and groups the related events.

Step 4: Review the evidence

The chunk appears in the dashboard. The operator edits or approves it during the ten-second window.

Step 5: Retrieve context

The RAG system retrieves related error logs, product documentation and previous incident records.

Step 6: Diagnose the problem

Gemma proposes a likely cause, identifies supporting evidence and recommends an engineering investigation.

Step 7: Resolve the issue

A simulated engineering fix is deployed to the demonstration website. A test verifies that the checkout action now responds successfully.

Step 8: Verify the resolution

Churnce records the release, successful test and relevant telemetry as resolution evidence.

Step 9: Generate a customer email

The system prepares a benefit-focused re-engagement email for eligible customers whose checkout journey was interrupted.

Step 10: Measure the outcome

A subsequent simulated journey demonstrates successful checkout. The dashboard records the intervention and its observed result.

The demonstration should clearly distinguish simulated results from measurements collected from real customers.

25. Known Limitations

- Browser observation cannot reliably determine every backend failure without additional application instrumentation.
- Semantic similarity does not establish causation.
- Language models may generate incorrect diagnoses or unsupported claims.
- Ten seconds is not sufficient for detailed human review of every chunk at scale.
- Real-time embedding and inference costs may become significant with high event volumes.
- Historical outcomes may be biased or insufficient for reliable intervention selection.
- A verified technical fix does not guarantee that every customer will experience an improvement.
- Customer churn may depend on external factors that the platform cannot observe.
- Email effectiveness depends on consent, deliverability, timing and customer preferences.
- Browser extension permissions and behavior differ across websites and browser environments.

These limitations should be handled through uncertainty reporting, source traceability, human oversight and controlled evaluation.

26. Future Development

- More sophisticated anomaly detection.
- Cross-session pattern detection with appropriate privacy safeguards.
- Integration with issue trackers and customer support platforms.
- Advanced causal experimentation.
- Automated prioritization based on engineering impact.
- Additional supported AI providers.
- On-premises deployment for sensitive environments.
- Customer journey replay using privacy-filtered events.

- Automated regression-test recommendations.
- Feature-demand analysis from repeated customer behavior.
- Controlled campaign experiments.
- More advanced intervention outcome modeling.
- Local or edge inference where suitable models and hardware are available.

These enhancements should follow a working and measurable core pipeline.

27. Expected Outcomes

A successful Churnce prototype should demonstrate that customer behavior can be transformed into actionable business intelligence through a single observable workflow.

- Detect customer friction earlier.
- Identify potential technical and usability root causes.
- Connect product quality to customer lifecycle outcomes.
- Prioritize evidence-backed improvements.
- Generate relevant, verified customer communications.
- Reduce unnecessary or poorly timed marketing interventions.
- Measure the outcomes of product and lifecycle improvements.
- Build an accumulating memory of incidents and interventions.

The most important outcome is not the number of emails generated or chunks indexed. It is whether Churnce helps the business understand and resolve the problems that prevent customers from succeeding.

28. Conclusion

Churnce combines browser-based customer observation, semantic chunking, a human-controllable ingestion gate, retrieval-augmented generation, Google Gemma, NVIDIA AI services and lifecycle marketing into a unified customer intelligence platform.

Its distinguishing feature is the connection between technical and experiential problems and the customer retention strategies that follow.

When a customer struggles with a product, Churnce helps identify the obstacle. When a business fixes it, Churnce verifies the improvement and prepares an appropriate next step. When that intervention produces measurable results, the system records the outcome for future analysis.

Rather than treating customer churn as a problem that can be solved by sending more messages, Churnce investigates the experience that caused disengagement in the first place.

CHURNCE: Understand the friction. Improve the experience. Bring customers back.

29. Reference Documentation

The following official resources should guide implementation. Exact API support and model availability should be confirmed when development begins.

Google Gemma documentation: https://ai.google.dev/gemma/docs

Google Gemini API documentation and hosted model access: https://ai.google.dev/gemini-api/docs

NVIDIA API catalog: https://build.nvidia.com/

NVIDIA NeMo Retriever documentation: https://docs.nvidia.com/nemo/retriever/

NVIDIA NIM API documentation: https://docs.api.nvidia.com/

Chrome Extensions documentation: https://developer.chrome.com/docs/extensions/

PostgreSQL documentation: https://www.postgresql.org/docs/

pgvector repository and documentation: https://github.com/pgvector/pgvector

FastAPI documentation: https://fastapi.tiangolo.com/

React documentation: https://react.dev/

Implementation note: Churnce is a proposed system architecture. The APIs, integrations, models, event sources and business outcomes described here must be implemented and tested before they can be considered operational capabilities.